

1. Implement Perceptron with Vector and Matrix:

A. transfer inputs and weights of perceptron into vector or matrix format can't decrease the sum of computing, but it enables users to take advantage of efficient mathematics libraries (ex: Torch/NumPy...), which use GPU to parallelly compute data for more efficiency.

B. vector format for input variables of single set of training example with $n+1$ input variables:

$$\vec{X} = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{bmatrix}_{(n+1) \times 1} \quad \& \quad \vec{X}^T = [x_0 \ x_1 \ \dots \ x_n]_{1 \times (n+1)}$$

→ single set of training example

C. vector format for weight variables of single perceptron with $n+1$ weight variables:

$$\vec{W} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{bmatrix}_{(n+1) \times 1} \quad \& \quad \vec{W}^T = [w_0 \ w_1 \ \dots \ w_n]_{1 \times (n+1)}$$

→ single perceptron

D. matrix format for input variables of $m+1$ set of training examples with $n+1$ input variables:

$$X_{(n+1) \times (m+1)} = \begin{bmatrix} x_{00} & x_{01} & \dots & x_{0n} \\ x_{10} & x_{11} & \dots & x_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m0} & x_{m1} & \dots & x_{mn} \end{bmatrix}_{(n+1) \times (m+1)}$$

→ $m+1$ set of training examples

$$X^T_{(m+1) \times (n+1)} = \begin{bmatrix} x_{00} & x_{01} & \dots & x_{0n} \\ x_{10} & x_{11} & \dots & x_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m0} & x_{m1} & \dots & x_{mn} \end{bmatrix}_{(m+1) \times (n+1)}$$

→ $m+1$ set of training examples

E. matrix format for weight variables of $m+1$ perceptrons with $n+1$ weight variables:

$$W_{(n+1) \times (m+1)} = \begin{bmatrix} w_{00} & w_{01} & \dots & w_{0n} \\ w_{10} & w_{11} & \dots & w_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m0} & w_{m1} & \dots & w_{mn} \end{bmatrix}_{(n+1) \times (m+1)}$$

→ $m+1$ perceptrons

$$W^T_{(m+1) \times (n+1)} = \begin{bmatrix} w_{00} & w_{01} & \dots & w_{0n} \\ w_{10} & w_{11} & \dots & w_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m0} & w_{m1} & \dots & w_{mn} \end{bmatrix}_{(m+1) \times (n+1)}$$

→ $m+1$ perceptrons

2. Vector-Vector (Dot-Product) Expression:

A. single perceptron with $n+1$ weight variables ($\vec{W}^T$) and single set of training example with $n+1$ input variables ($\vec{X}$)

B. $z = \vec{W}^T \cdot \vec{X}$
number of value z : 1

3. Matrix-Vector Expression:

A. $m+1$ perceptron with $n+1$ weight variables ($W^T_{(m+1) \times (n+1)}$) and single set of training example with $n+1$ input variables ($\vec{X}$)

B. $z = W^T_{(m+1) \times (n+1)} \cdot \vec{X}$
number of value z : $m+1$

4. Matrix-Matrix Expression:

A. $m+1$ perceptron with $n+1$ weight variables ($W^T_{(m+1) \times (n+1)}$) and $p+1$ set of training example with $n+1$ input variables ($X_{(n+1) \times (p+1)}$)

B. $z = W^T_{(m+1) \times (n+1)} X_{(n+1) \times (p+1)}$
number of value z : $(m+1) \times (p+1)$

5. Concept of Dimension:

A. scalar: 0 dimension data structure.

B. vector: 1 dimension data structure.

C. matrix: 2 dimension data structure.

D. tensor: 3+ dimension (multidimension) data structure.